



Database Implementation & Technical Documentation

PostgreSQL 18 • Spring Boot API • Claims Management

Prepared for the Claimly development team

Database: claimly

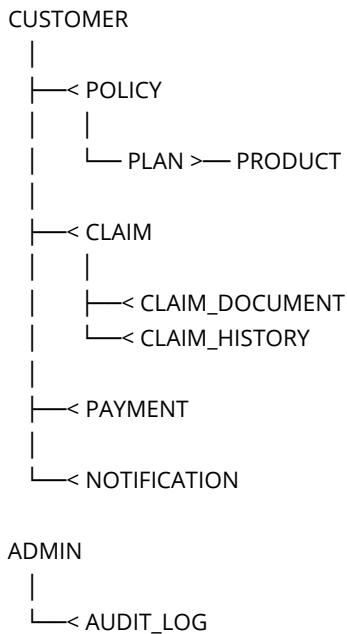
Schema: public

1. Purpose of the Database

The Claimly database is the persistent data layer for the insurance claims platform. It supports the complete core workflow: customer registration, insurance products and plans, policy management, claim submission, supporting documents, claim-status history, payments, customer notifications and administrative auditability.

The database is implemented in PostgreSQL 18 and is connected to the Spring Boot API through Spring Data JPA. The running API connects to the local claimly database at jdbc:postgresql://localhost:5432/claimly.

2. Overall Relationship



The '<' notation means one-to-many. Foreign keys enforce these relationships at database level.

3. Final Tables and Responsibilities

Table	Purpose	Key relationship
customers	Customer identity, contact information and account status.	Customer → policies, claims, payments, notifications
products	Insurance product definitions.	Product → plans
plans	Product tiers, premiums and benefit rules.	Plan → product; policies → plan
policies	Connects a customer to a selected insurance plan.	Policy → customer and plan
claims	Claim submission, lifecycle and payout information.	Claim → customer and policy
claim_documents	Supporting claim-document metadata and file location.	Document → claim
claim_history	Status-change history for claims.	History → claim
payments	Customer payments against policies.	Payment → customer and policy
notifications	Customer communication and delivery status.	Notification → customer
audit_logs	Traceability of administrative/system actions.	Audit log → admin/action/entity

4. Core Implementation

4.1 Customers

The customers table is the starting point for the customer insurance workflow. SA ID number and phone number are unique to prevent duplicate records.

```
customers
-----
id          BIGSERIAL PRIMARY KEY
full_name   VARCHAR(150) NOT NULL
sa_id_number VARCHAR(13) NOT NULL UNIQUE
phone_number VARCHAR(20) NOT NULL UNIQUE
status      VARCHAR(30) NOT NULL DEFAULT 'ACTIVE'
created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

4.2 Products and Plans

Products define the insurance offering, while plans define selectable tiers within a product. The UNIQUE(product_id, tier) constraint prevents duplicate tiers under the same product.

```
PRODUCT
├── PLAN
│   ├── tier
│   ├── premium
│   ├── benefit_amount
│   ├── benefit_duration
│   └── waiting_period
```

4.3 Policies

A policy connects a customer to a selected plan. policy_number is a unique business identifier. This lets the API determine the plan and benefits applicable to a customer's policy.

4.4 Claims

Claims connect customers to policies and store the claim lifecycle. claim_reference is a unique customer-facing identifier.

```
SUBMITTED
↓
UNDER_REVIEW
↓
APPROVED
↓
PAID
```

```
Alternative:
UNDER_REVIEW
↓
DECLINED
↓
Reason / next steps
```

4.5 Claim Documents

One claim can have many documents. The database stores file metadata and file_url, keeping file storage separate from the claim record.

4.6 Claim History

claim_history records old_status, new_status, who changed it, the time and the reason. This supports the claim-tracking requirement and provides a business timeline.

4.7 Payments and Notifications

Payments are linked to both customer and policy, allowing payment history to be retrieved by either context. Notifications are linked to customers and can represent claim updates, payment reminders and general communication.

5. Data Integrity Controls

Control	Implementation	Purpose
Primary keys	BIGSERIAL PRIMARY KEY	Unique row identification.
Foreign keys	REFERENCES ...	Validates table relationships.
Unique identifiers	UNIQUE constraints	Prevents duplicate SA IDs, phone numbers, policy numbers and claim references.
Required fields	NOT NULL	Prevents incomplete mandatory records.
Financial checks	CHECK >= 0	Prevents negative premium, benefit, payment and file-size values.
Delete rules	RESTRICT / CASCADE	Controls dependent-record deletion.

6. Indexing

Indexes were added to foreign-key columns and frequently queried claim fields to support customer lookups, policy searches, claim filtering and administration.

Indexed area	Purpose
plans.product_id	Retrieve plans for a product.
policies.customer_id / plan_id	Retrieve customer and plan policies.
claims.customer_id / policy_id	Retrieve customer and policy claims.
claims.status	Filter claims by lifecycle status.
claim_documents.claim_id	Retrieve documents for a claim.
claim_history.claim_id	Retrieve a claim timeline.
payments.customer_id / policy_id	Retrieve payment history.
notifications.customer_id	Retrieve customer notifications.
audit_logs.entity, entity_id	Find actions for a specific business record.

7. Audit Logging – Improved Design

The original audit_logs table is a good baseline, but the project requirement is stronger than simply storing an action and entity. For administrative accountability, the system should record who performed the action, what record was affected, what changed, when it changed and why.

Recommended administrator table:

```
CREATE TABLE admins (  
    id BIGSERIAL PRIMARY KEY,
```

```

full_name VARCHAR(150) NOT NULL,
email VARCHAR(255) NOT NULL UNIQUE,
status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

Recommended audit_logs table:

```

CREATE TABLE audit_logs (
  id BIGSERIAL PRIMARY KEY,

  admin_id BIGINT NOT NULL,

  action VARCHAR(100) NOT NULL,
  entity VARCHAR(100) NOT NULL,
  entity_id BIGINT,

  old_value JSONB,
  new_value JSONB,

  reason TEXT,

  ip_address INET,
  user_agent TEXT,

  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT fk_audit_logs_admin
    FOREIGN KEY (admin_id)
    REFERENCES admins(id)
    ON DELETE RESTRICT
);

```

```

CREATE INDEX idx_audit_logs_admin_id
  ON audit_logs(admin_id);

```

```

CREATE INDEX idx_audit_logs_entity
  ON audit_logs(entity, entity_id);

```

```

CREATE INDEX idx_audit_logs_created_at
  ON audit_logs(created_at);

```

The main improvement is old_value and new_value using JSONB. This allows the system to record the relevant before-and-after values without redesigning the audit table every time another business field is introduced.

Field	Meaning	Example
admin_id	Administrator responsible for the action.	17
action	Operation performed.	CLAIM_STATUS_CHANGED
entity	Business object affected.	CLAIM
entity_id	Affected record ID.	1042
old_value	Value before the action.	{"status":"UNDER_REVIEW"}

new_value	Value after the action.	{"status":"APPROVED"}
reason	Explanation for the action.	Documents verified
ip_address	Source address where available.	192.168.1.20
user_agent	Client information where available.	Browser/client
created_at	Time recorded.	2026-08-05 11:30:00

8. Example: Administrative Claim Approval

```

action = 'CLAIM_STATUS_CHANGED'
entity = 'CLAIM'
entity_id = 1042
old_value = {"status":"UNDER_REVIEW"}
new_value = {"status":"APPROVED"}
reason = 'Required supporting documents verified'

```

This provides the Admin Claims Console with a traceable record of the administrator, claim, previous status, new status, reason and timestamp.

9. Claim History vs Audit Logs

Component	Question it answers
claim_history	How did the claim status change over its lifecycle?
audit_logs	Who performed an administrative/system action, what record was affected, what changed and why?

They are complementary rather than duplicate tables. A claim-status update performed by an administrator can create a claim_history record for the business timeline and an audit_logs record for accountability.

10. Foreign-Key Verification

The relationship query currently used in pgAdmin verifies the implemented foreign keys in the public schema:

```

SELECT
    tc.table_name,
    kcu.column_name,
    ccu.table_name AS referenced_table,
    ccu.column_name AS referenced_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
    AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
    AND ccu.table_schema = tc.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
    AND tc.table_schema = 'public'
ORDER BY tc.table_name, kcu.column_name;

```

The current verification confirms relationships for claim_documents → claims, claim_history → claims, claims → customers/policies, notifications → customers, payments → customers/policies, plans → products, and policies → customers/plans.

11. Spring Boot Integration

The database maps directly to Spring Data JPA entities. The running API is using Spring Boot 4.0.7, Hibernate ORM 7.2.19.Final and PostgreSQL JDBC, and starts on port 8080.

```
src/main/java/com/Claimly
|
|— customer
|— product
|— plan
|— policy
|— claim
|   |— Claim.java
|   |— ClaimDocument.java
|   |— ClaimHistory.java
|   |— ClaimRepository.java
|   |— ClaimService.java
|   |— ClaimController.java
|— payment
|— notification
|— audit
|   |— Admin.java
|   |— AuditLog.java
|   |— AuditLogRepository.java
|   |— AuditLogService.java
```

12. Recommended JPA Relationship Mapping

Database relationship	JPA mapping
Customer → Policies	@OneToMany / @ManyToOne
Product → Plans	@OneToMany / @ManyToOne
Plan → Policies	@OneToMany / @ManyToOne
Customer → Claims	@OneToMany / @ManyToOne
Policy → Claims	@OneToMany / @ManyToOne
Claim → Documents	@OneToMany / @ManyToOne
Claim → History	@OneToMany / @ManyToOne
Customer → Payments	@OneToMany / @ManyToOne
Policy → Payments	@OneToMany / @ManyToOne
Customer → Notifications	@OneToMany / @ManyToOne
Admin → Audit Logs	@OneToMany / @ManyToOne

13. Team Implementation Notes

- The database should remain the source of truth for primary keys, foreign keys, uniqueness and basic data validation.
- The Spring Boot service layer should enforce business rules such as permitted claim-status transitions and authorization.

- Claim status changes should create claim_history records and, for administrative changes, audit_logs records.
- admin_id should be derived from the authenticated administrator/security context rather than trusted as arbitrary client input.
- Claim documents store metadata and file_url; actual file storage remains separate from the relational claim record.
- The core MVP schema is already implemented; the audit enhancement strengthens administrator accountability without changing the main customer-policy-claim workflow.

14. Implementation Status

Area	Status
PostgreSQL 18 / claimly database	Implemented
Customers	Implemented
Products and plans	Implemented
Policies	Implemented
Claims	Implemented
Claim documents	Implemented
Claim history	Implemented
Payments	Implemented
Notifications	Implemented
Foreign-key relationships	Implemented and verified
Indexes	Implemented
Audit-log baseline	Implemented
Admin-linked audit logging	Recommended improvement
JSONB change tracking	Recommended improvement

15. Conclusion

The Claimly PostgreSQL database provides a normalized foundation for the claims-management API. It separates customer, product, plan, policy, claim, document, history, payment, notification and audit responsibilities, while foreign keys, constraints and indexes protect integrity and support the main application workflows.

The recommended audit enhancement introduces an explicit admins table, links audit_logs.admin_id to the administrator, and records old/new values as JSONB. This directly strengthens traceability for the Admin Claims Console and makes administrative claim-status changes auditable.

pgAdmin 4

File Object Tools Edit View Window Help

Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (2)
 - claimly
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

Dashboard x Properties x SQL x Statistics x Dependencies x Dependents x Processes x claimly/postgres@PostgreSQL 18

claimly/postgres@PostgreSQL 18

No limit

Query Query History

Scratch Pad x

```
1  -- =====
2  -- CLAIMLY DATABASE
3  -- PostgreSQL 18
4  -- =====
5  ---This is the Customer Table
6  -- =====
7  -- CUSTOMERS
8  -- =====
9
10 CREATE TABLE customers (
11     id BIGSERIAL PRIMARY KEY,
12     full_name VARCHAR(150) NOT NULL,
13     sa_id_number VARCHAR(13) NOT NULL UNIQUE,
14     phone_number VARCHAR(20) NOT NULL UNIQUE,
15     status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',
16     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
17     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
18 );
19 ---This is the product Table
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	table_name	column_name	referenced_table	referenced_column
	name	name	name	name
1	claim_documen...	claim_id	claims	id
2	claim_history	claim_id	claims	id
3	claims	customer_id	customers	id
4	claims	policy_id	policies	id
5	notifications	customer_id	customers	id
6	payments	customer_id	customers	id
7	payments	policy_id	policies	id
8	plans	product_id	products	id
9	policies	customer_id	customers	id
10	policies	plan_id	plans	id

Total rows: 10 Query complete 00:00:00.049 CRLF Ln 310, Col 19

20°C Sunny 1:13 PM 8/5/2026